

Microservices

INF 4040 TECHNOLOGIES FOR WEB-BASED INFORMATION SYSTEMS

AUBG COS DEPARTMENT

Web Services

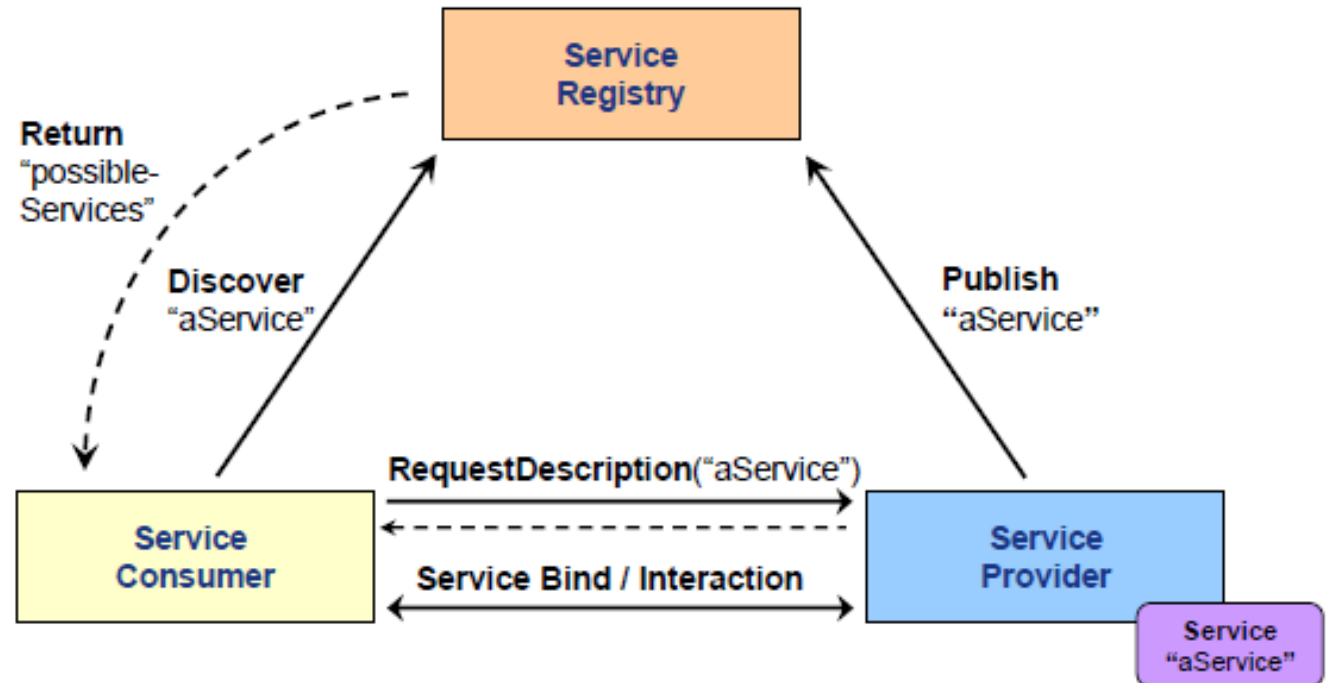
- Many companies have data/resources that they are willing to share with the public and other companies
- These organizations are providing **a service** to the public and other companies by sharing their data/resources
- When such services are provided across Internet, i.e. the Web, they are called **Web services**
- Google, Amazon, etc. are examples of companies offering such Web services
 - E.g. Google Maps

Web Services (continued)

- The services are accessed via published **APIs** that the companies/organizations advertise
 - E.g. Google Maps – a number of APIs for different mapping services
- The main use for Web services is **between computers**, rather than a person and a computer
- The main **data formats** used are **XML** and **JSON** – service results are delivered in these formats
- The “umbrella” term for this software architectural style is a **Service-Oriented Architecture (SOA)**

Web Service-Oriented Architecture Model

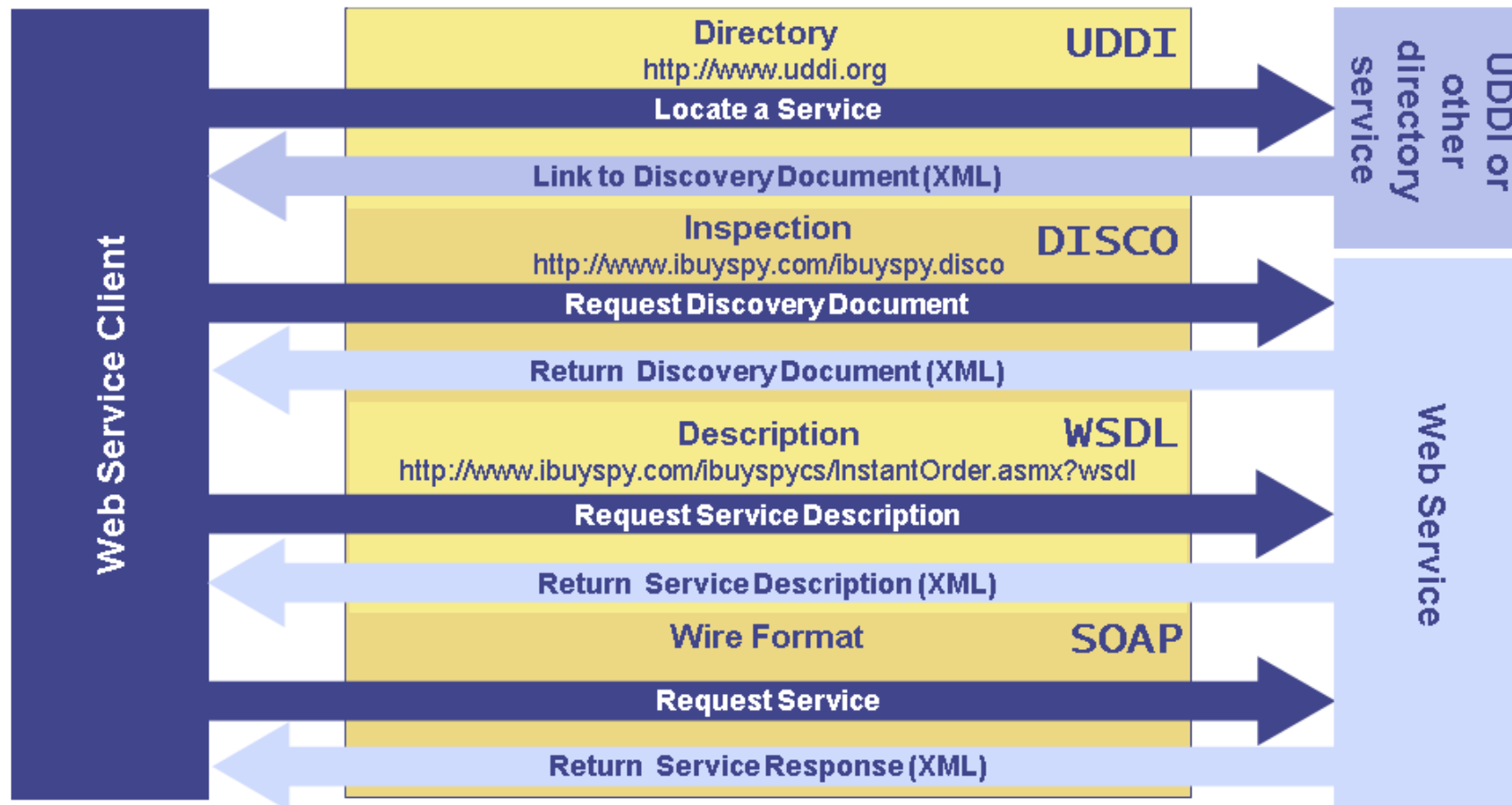
1. **Provider** publishes details of its service in the registry
2. **Consumer** queries registry for a service provider
3. **Consumer** contacts **provider** to negotiate delivery of service and bind with the provider



First Era of Providing Web Services

- Around 2000, IBM, Microsoft et al. developed a set of XML-based protocols for the implementation of Web Services
 - SOAP, XML-RPC
- These protocols use HTTP for underlying transport
- But they were **very heavyweight**, and **cumbersome to use**
 - Built collaboratively by several development teams
 - With a traffic load that requires horizontal scaling (i.e. load balancing across multiple copies of the system)

XML-Based Web Service Technologies

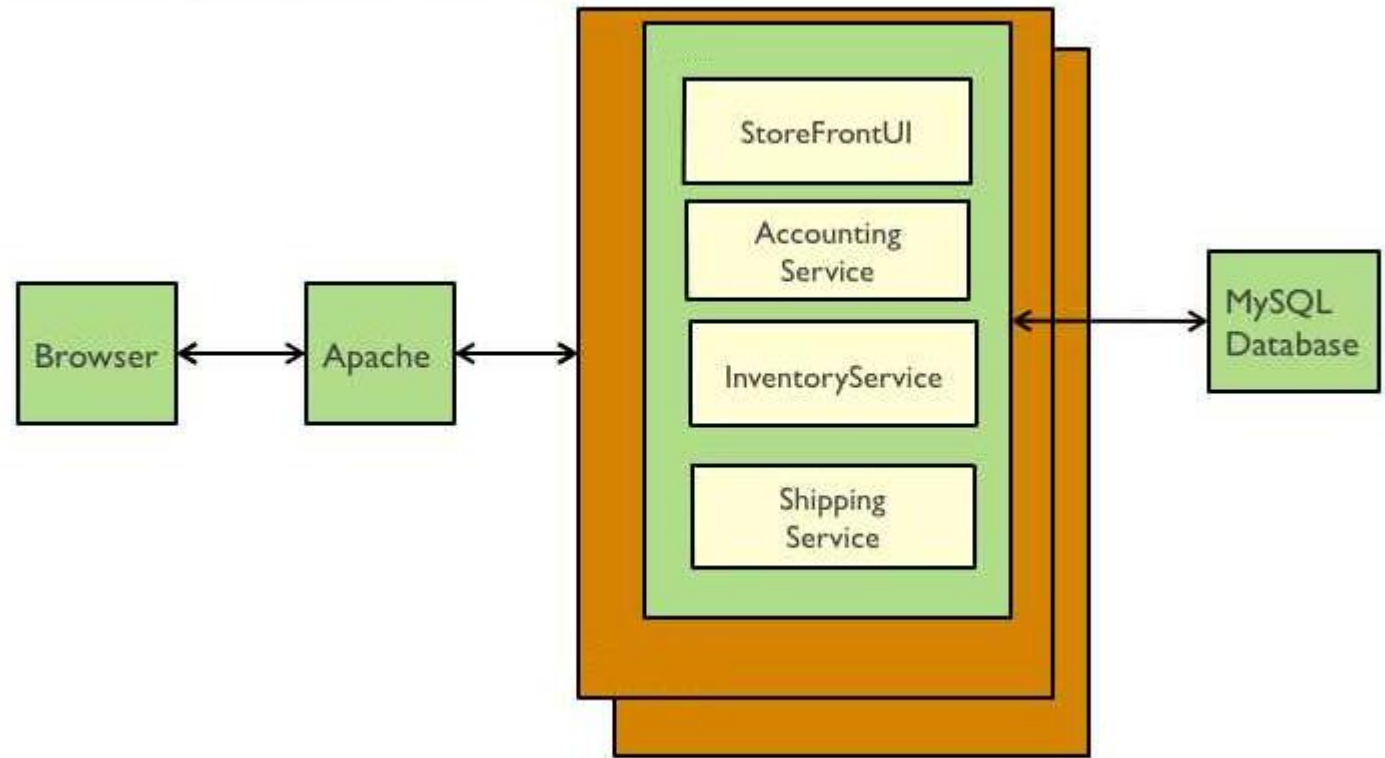


Second Era of Providing Web Services

- The introduction of the **RESTful** design style for networked apps added a new option for the provision of Web services
- Slowly RESTful-based Web services replaced the first-era heavyweight protocols
- **REST over HTTP** is very lightweight compared to the XML-based software stack
 - Much easier to use – HTTP-based CRUD operations
- However, Web services technology **still suffered from problems**
 - Principally, the design of the **Web services architecture**

Current Situation (But Changing)

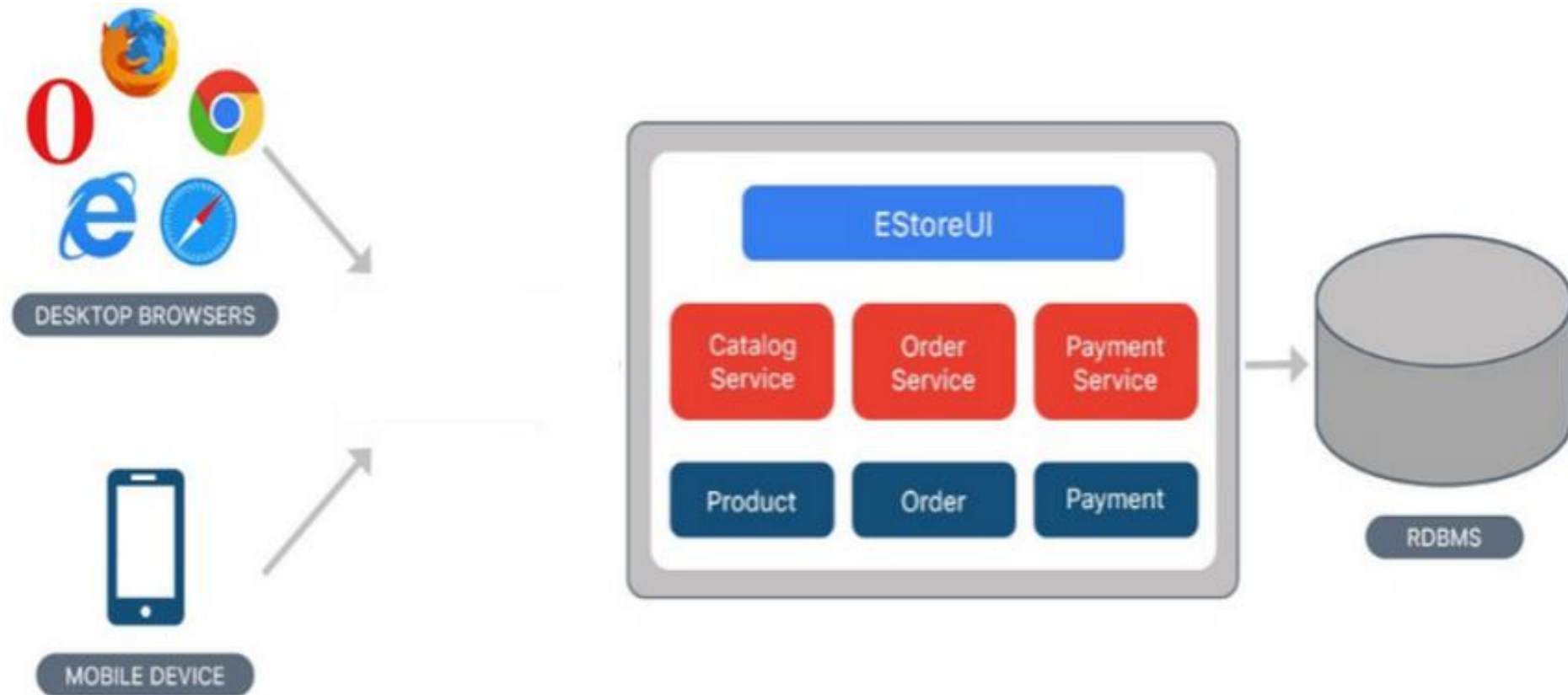
- Web services were traditionally developed as a “monolithic” structure
- The different services that may be offered by a single enterprise were packaged together as a **single unit**
- E.g.: a **single API** for lots of different services



Monolith Architecture

- A monolithic application is **self-contained** and **independent** from other applications
 - Example – an application in which the user interface, the business logic and data access code are combined into a **single program**
- **Design philosophy** is that the application is responsible not just for a particular task, but can perform every step needed to complete a particular function
 - Definitely a **heavy** design philosophy

Monolithic Architecture Example: e-Commerce Application



Characteristics of a Software Monolith

- One build and deployment unit
- One code base
- One technology stack, e.g. ASP.NET Framework
- Benefits
 - Simple mental model for developers – one unit of access for coding, building, and deploying
 - Simple scaling model for operations – running multiple copies behind a **load balancer**

Problems with Software Monoliths

- Huge and intimidating code base for developers
- Development tools get overburdened
 - Refactoring take minutes, builds take hours, Continuous Integration testing takes days
- Scaling is limited
 - Running a copy of the whole system is resource-intensive
 - It does not scale with the data volume, out-of-the-box
- Deployment frequency is limited
 - Re-deploying means halting the whole system and may possibly fail

Challenges with monolithic software

Difficult to
scale

Architecture is
hard to maintain
and evolve

Lack of agility

Long
Build/Test/Release
Cycles
(who broke the build?)

New releases
take months

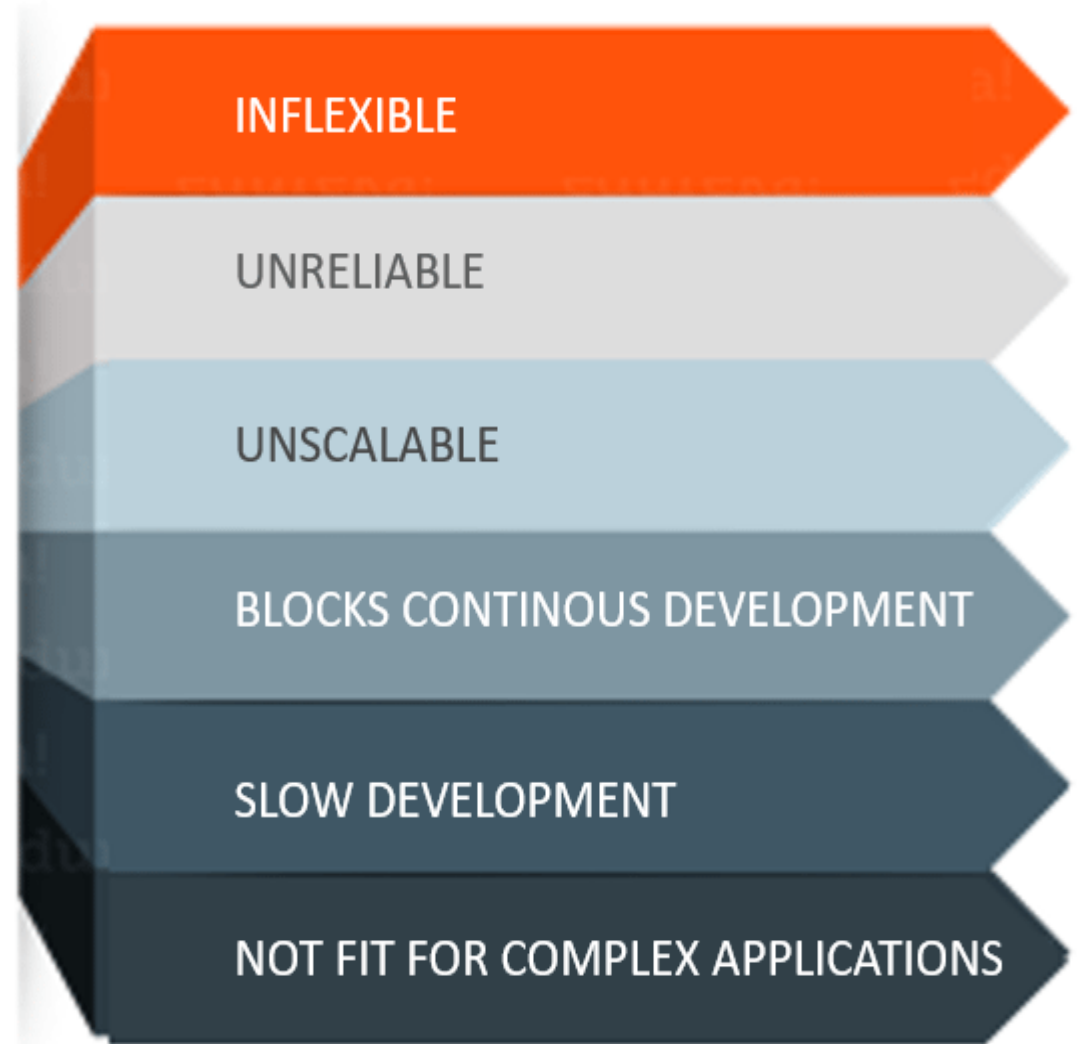
Lack of innovation

Operations
is a nightmare
(module X is failing,
who's the owner?)

Long time to add
new features

Frustrated customers

Downsides of Monolithic Structures



Third (Modern) Era of Providing Web Services

- The monolith is “broken up” into a set of **independent microservices** that are deployed and maintained **separately**
- A **microservice** is an architectural style for web services that structures an application as a collection of **loosely-coupled** (aka **autonomous**) **services**
- The word “micro” refers to the sizing
 - One microservice must be manageable by a single development team (5-8 developers)

Microservices Advantages

- Services are encouraged to be small – less time to develop, and easy to deploy
- Services, using a standard interface protocol, can be “consumed” and re-used by other services, e.g. Mashups – Trivago
- Services exist as independent deployment artifacts, and can be scaled independently

Microservice Properties

- Each **microservice** is functionally complete with
 - Resource representation
 - Data management
- Each **microservice** provides just one service/resource – one service per URI
 - E.g. A different microservice for Clients, Catalog, Shopping Cart, Invoicing, etc
- Some industry experts now refer to a **MicroService Architecture (MSA)** rather than a **SOA**

Some Definitions of Microservices

- Small and focused on doing one thing well
- Autonomous

"... services are independently deployable and scalable, each service also provides a firm module boundary, even allowing for different services to be written in different programming languages"

"Loosely coupled service oriented architecture with bounded contexts"

Adrian Cockcroft (Netflix)

"SOA done right"

Microservices

**"service-oriented
architecture**

composed of
loosely coupled
elements
that have
bounded contexts"

Services communicate with
each other over the
network

Microservices (continued)

"service-oriented
architecture
composed of
**loosely coupled
elements**
that have
bounded contexts"

You can update the services independently; updating one service doesn't require changing any other services.

Microservices (continued)

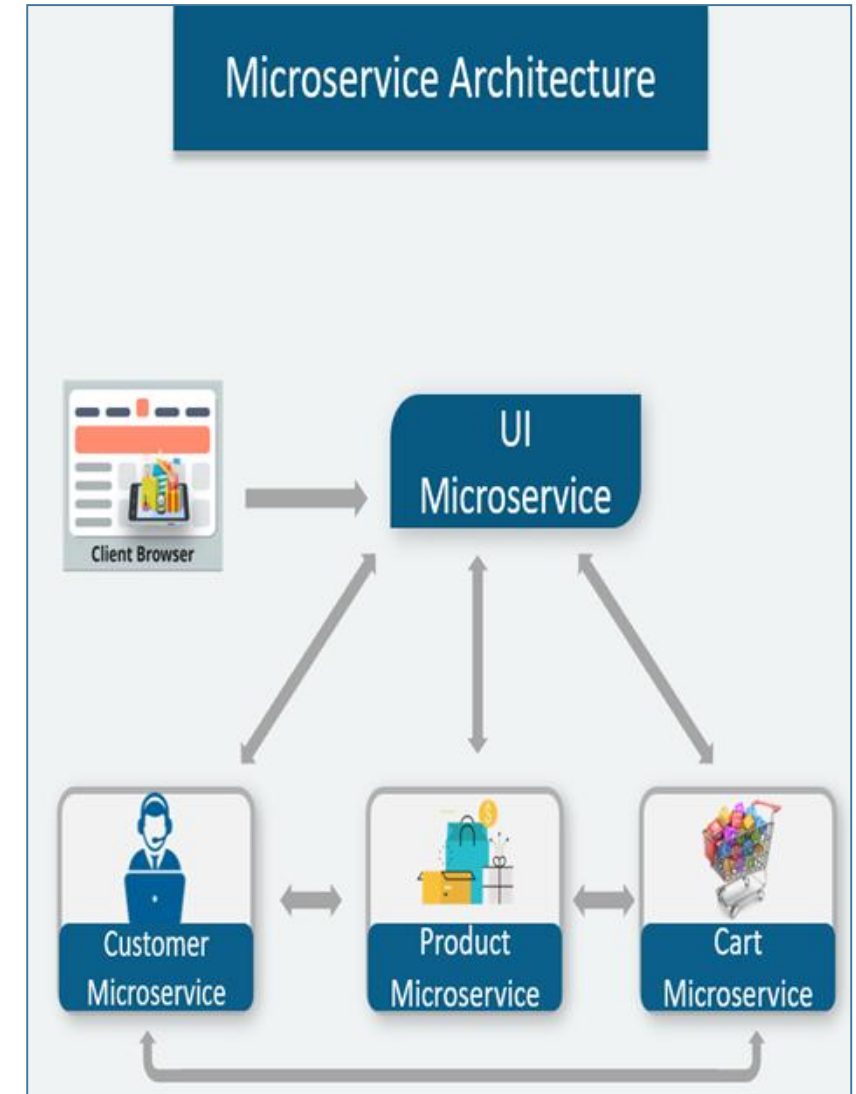
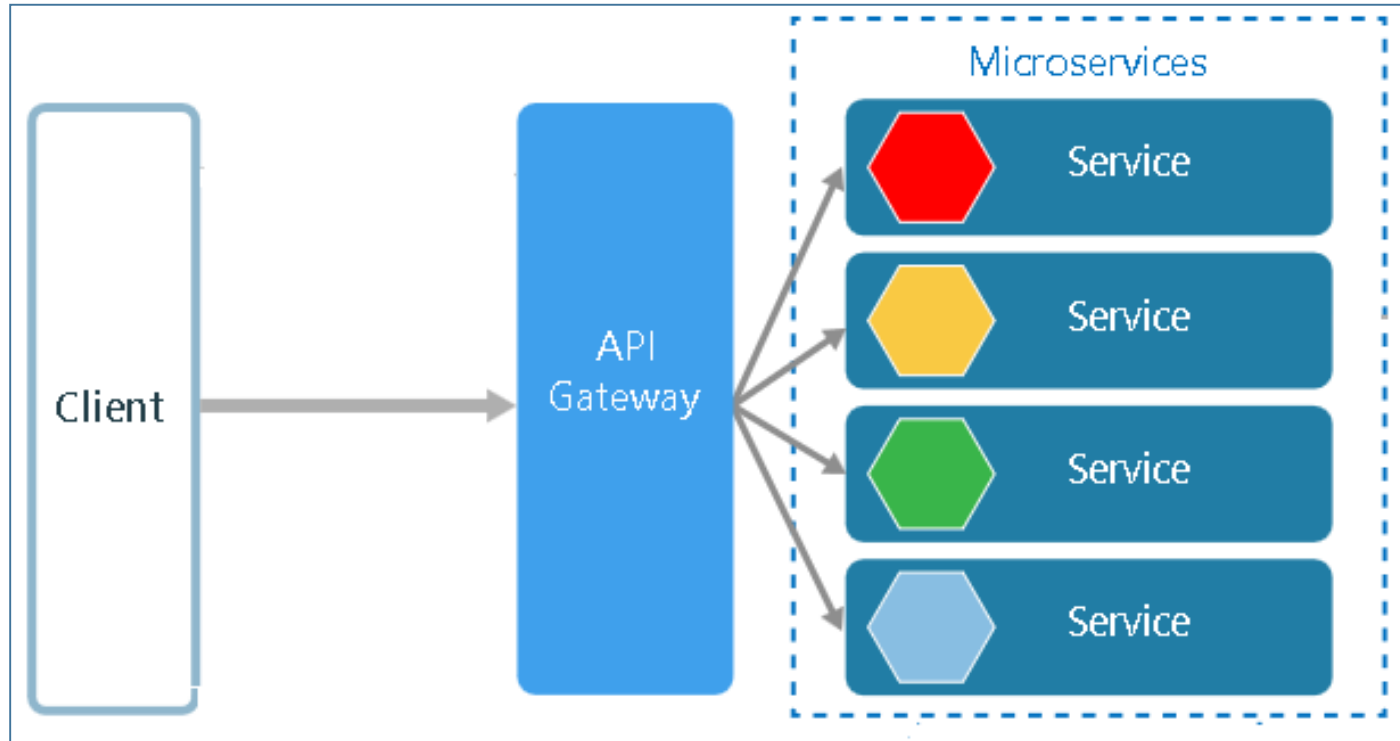
"service-oriented
architecture
composed of
loosely coupled
elements
that have
bounded contexts"

Self-contained; you can
update the code without
knowing anything about the
internals of other
microservices

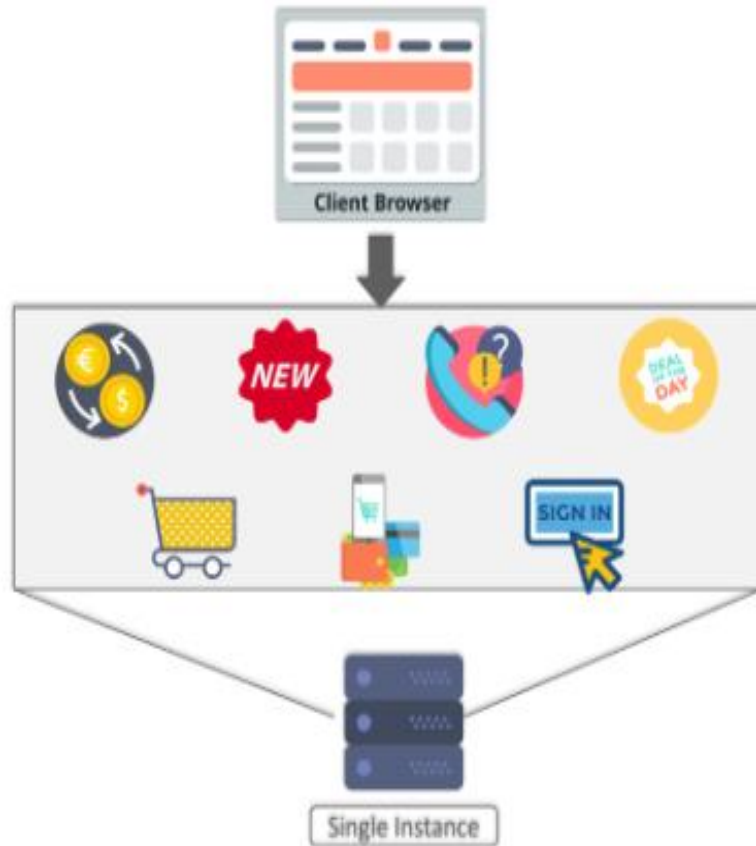
Microservice Architecture

- Essentially, microservices follow well-known software development practices
 - Separation of Concerns
 - Single Responsibility
 - Separate Interfaces

Microservice Architectures

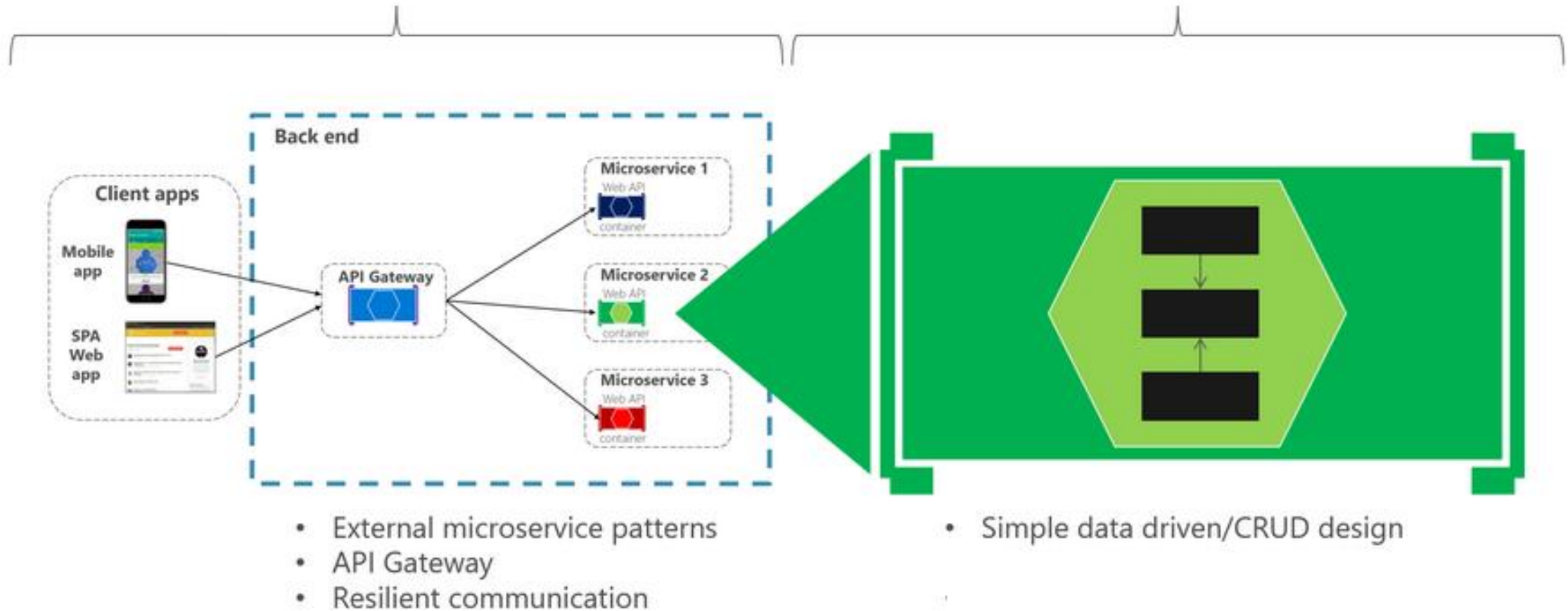


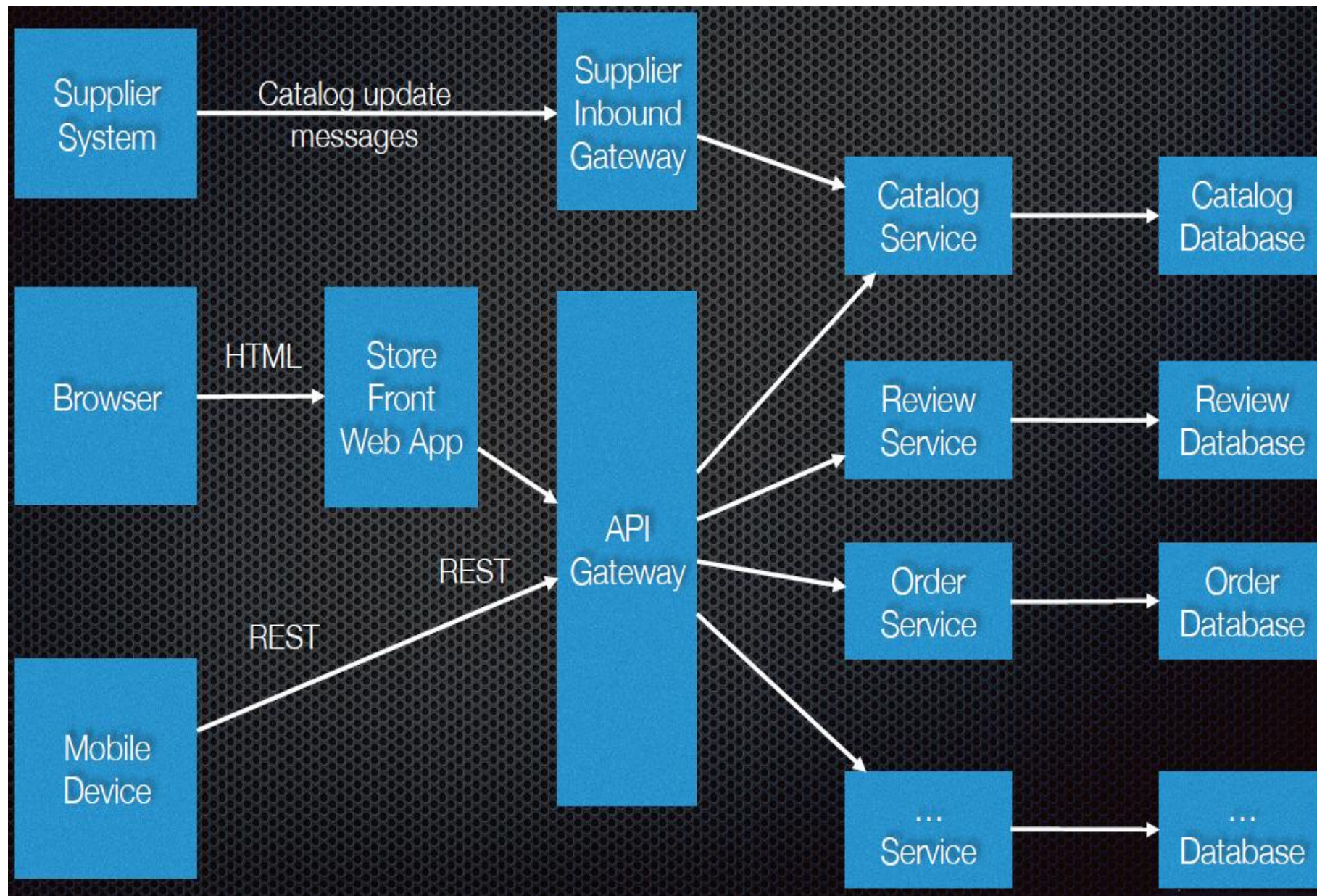
Monolithic vs Microservices



External architecture per application

Internal architecture per microservice





Microservices Examples

- **Netflix** has a widespread architecture that has evolved from monolithic to microservices. It receives more than **one billion calls every day**, from more than **800 different types of devices**, to its **streaming-video API**. Each API call makes around five additional calls to the backend service
- **Amazon** has also migrated to microservices. They get countless calls from a variety of applications – including Amazon's own applications that manage the web service API as well as the website itself – which would have been simply impossible for their old monolithic architecture to handle
- The auction site **eBay** is yet another example that has gone through the same transition. Their core application comprises several autonomous applications, with **each one executing the business logic for different function areas**

Benefits of microservices

Easier to scale
each
individual
micro-service

Easier to
maintain and
evolve system

Increased agility

Rapid
Build/Test/Release
Cycles

New releases
take minutes

Faster innovation

Clear ownership and
accountability

Short time to add
new features

Delighted customers

Service Discovery

- **Service discovery** is about finding the network location of a service provider
- Need to maintain **a list of services** and which **servers** they are located on
 - Need to enable **service lookup** to find the **endpoint** for a service – e.g. IP address and port number
- In a simple architecture, a static configuration works pretty well
 - A service is **always** deployed at the **same location**
- But this is not the case in microservices architectures

Service Discovery (continued)

- One objective of microservices is to enable the development teams to deploy and scale their applications **independently**
- As more services and instances are added to the system, service locations may start to **change frequently** – partly because of manual or automatic scaling, partly because of continuous delivery processes
- Clearly, a static configuration of the location of services is not a viable solution
 - A **dynamic configuration** is needed

Components of Service Discovery

- Service discovery involves three components – **service providers**, **service registry**, and **service consumers**
 - **Service providers** register themselves with a **service registry** when they enter service, and deregister themselves when they leave
 - **Service registry** maintains the latest location of **service providers**
 - **Service consumers** get the location of a **service provider** from a **service registry**, and then communicate with the **service provider**

Service Discovery Packages

- There are many software packages suitable for **service discovery**
 - E.g., **Apache ZooKeeper** for cloud-based services
- But the current favorite is **Netflix's Eureka**
 - A **Eureka server** – provides a service discovery registry, which is a database of services, their instances and their locations. Can be accessed by clients. Services must register their APIs
 - A **Eureka client** – integrates into the client application – makes requests to a Eureka server. Querying the service registry to find the available instances of a service

Methods of Service Discovery

- There are two methods for service discovery
- **Client-side discovery** – client-side logic picks the service instance to send a request to
- **Server-side discovery** – client sends a request to a router that picks the service instance to which to forward the request

Microservice Problems

- The tradeoff of microservices' flexibility is **complexity**
 - Managing a multitude of distributed services at scale is difficult
 - Services potentially may become too fine-grained, i.e. too small
- The **performance can degrade** if too many services are used

Questions?

Q & A